

PROJECT REPORT

Complex Calculator

A Python Command-Line Arithmetic Application

Project Type	Console-based software application
Technology	Python 3
Source File	vityarthi-pro1.py
Name	Kanha Sanjay Pawar
Registration Number	26BAS10014
Session	2026
Published On	GitHub
Prepared	September 2026

1. Introduction

The Complex Calculator is a menu-driven Python command-line program for performing arithmetic calculations. It supports addition, subtraction, multiplication, division, and exponentiation. The program accepts multiple numeric operands for the four basic operations, validates user input, reports selected errors, and continues operating until the user chooses to exit.

The implementation is intentionally lightweight: it uses standard Python functionality, has no external dependencies, and does not persist data. Its primary value is educational, demonstrating functions, loops, conditional branching, exception handling, and interactive input/output.

2. Problem Statement

Users need a simple interactive utility that can evaluate common arithmetic expressions without requiring a graphical interface or complex setup. The system must accept numbers reliably, apply the selected operation in the expected order, handle invalid entries, prevent division by zero, and provide a clear exit path.

Scope. The current solution is a local, single-user console application. It does not include a graphical interface, expression parsing, calculation history, persistent storage, authentication, or advanced scientific functions.

3. Functional Requirements

- Display a welcome menu when the application starts and after each completed top-level cycle.
- Allow the user to select basic arithmetic, exponentiation, or exit.
- For basic arithmetic, allow selection of +, -, *, /, or q to leave the arithmetic submenu.
- Accept numbers one at a time until the user enters E or e.
- Perform addition across all supplied numbers.
- Perform left-to-right subtraction across all supplied numbers.
- Perform multiplication across all supplied numbers.
- Perform left-to-right division and detect a zero divisor.
- Accept a base and a power for exponentiation.
- Reject non-numeric input with an error message and continue interaction.
- Reject an operation when no number has been entered.
- Terminate with a thank-you message when the exit option is selected.

4. Non-functional Requirements

- Usability: prompts and menu options should be understandable to a novice user.
- Reliability: malformed numeric input should not terminate the program unexpectedly.
- Maintainability: arithmetic is separated into small functions.
- Portability: the program should run in a standard Python 3 environment.
- Performance: calculations should complete interactively for normal input sizes.
- Availability: no network, database, or external service should be required.
- Safety: division by zero should be reported rather than executed.

5. System Architecture

The system uses a simple layered logical architecture. The presentation layer manages prompts and printed results. The control layer implements the main menu loops and dispatches operations. The calculation layer contains add,

sub, multi, and div functions, while exponentiation is currently evaluated in the control flow. There is no persistence layer.

Architecture View

User	Enters menu choices, operators, numbers, and exit commands.
CLI / Presentation	input() collects values; print() displays menus, errors, and results.
Control Logic	while loops, if/elif branches, validation, and operation dispatch.
Calculation Functions	add(numbers), sub(numbers), multi(numbers), div(numbers).
Runtime	Python interpreter; standard library only; no external storage.

6. Design Diagrams

Use Case Diagram

Actor: User	Interacts with the calculator.
Use case: Select operation	Chooses basic arithmetic, exponentiation, or exit.
Use case: Enter operands	Provides one or more numeric values.
Use case: Calculate result	System applies the chosen operation.
Use case: Handle invalid input	System displays an error and requests another value.
Use case: Exit	User leaves the submenu or terminates the application.

Workflow Diagram

Start	Launch program.
Menu	Display options 0, 1, and 2.
Choice 0	Select operator; collect numbers until E/e; validate; calculate; print result; return to menu.
Choice 1	Read base and power; calculate base ** power; print result; return to menu.
Choice 2	Print farewell and terminate.
Other value	Print invalid-input message and repeat.

Sequence Diagram

1. User → CLI	Selects an operation.
2. CLI → User	Requests operands.
3. User → CLI	Enters values or E/e.
4. CLI → Calculator	Calls the relevant arithmetic function.
5. Calculator → CLI	Returns a numeric result or division-by-zero error.
6. CLI → User	Prints the result or error.

Class / Component Diagram

Calculator Functions	add(list), sub(list), multi(list), div(list). Stateless functional components.
----------------------	--

Main Controller	Top-level and arithmetic-menu loops; dispatches functions.
Input Validator	int conversion for menus; float conversion for operands; ValueError handling.
Output Interface	Console print statements for prompts, results, and errors.

ER Diagram / Storage

Storage used?	No.
Implication	There are no entities, relationships, tables, files, or persistent calculation records in the current implementation.
Optional future model	Calculation(id, operation, operands, result, timestamp) could support history.

7. Design Decisions & Rationale

- Command-line interface was selected because it is simple, portable, and suitable for demonstrating core programming concepts.
- Separate arithmetic functions improve readability and allow individual functions to be tested independently.
- Lists of operands support multi-number addition, subtraction, multiplication, and division rather than limiting the tool to two values.
- float conversion allows decimal input and produces more flexible results than integer-only conversion.
- Nested loops provide repeated use without restarting the program and allow a separate quit command for the arithmetic submenu.
- Exception handling around numeric conversion prevents common user mistakes from crashing the session.
- No database was introduced because the stated behavior does not require history or persistence.

8. Implementation Details

8.1 Arithmetic functions

add(number) uses Python sum() to total the list. sub(number) starts with the first item and subtracts each remaining item. multi(number) starts at 1 and multiplies every item. div(number) starts with the first item, checks each later item for zero, and divides sequentially.

8.2 Input and control flow

The outer while loop repeatedly displays the main menu. The basic-operation branch opens a second loop for operator selection. A third loop collects operands until E or e is entered. Invalid numeric input is caught through ValueError and does not append a value to the operand list.

8.3 Notable implementation issue

In the supplied source, the exponentiation and exit branches appear aligned under the basic-operation flow after the basic-operation result handling. In a runnable Python file, these branches should be aligned with the top-level operation selection so that choices 1 and 2 are reached correctly. Python indentation is therefore an important implementation correction before execution.

9. Screenshots / Results

No runtime screenshots were included with the source submission. The expected console results are illustrated below as representative test transcripts.

```

-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 0

```

```

operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: +
Enter a number(or 'E' to exit) 2
Enter a number(or 'E' to exit) 3
Enter a number(or 'E' to exit) E
result= 5.0

```

For a final submission, capture screenshots showing: the main menu, a successful calculation, invalid input handling, division by zero, exponentiation, and program exit.

10. Testing Approach

Testing should combine unit testing of the arithmetic functions with manual integration testing of the interactive loops. Because the current program reads directly from `input()`, interactive cases are best executed manually or through mocked input in an automated test harness.

Test ID	Scenario	Input	Expected result
T01	Addition	2, 3, 5	10.0
T02	Subtraction	10, 3, 2	5.0
T03	Multiplication	2, 4, 3	24.0
T04	Division	20, 4, 2	2.5
T05	Division by zero	10, 0	Error: can't divide by 0.
T06	No operands	E immediately	No number was entered.
T07	Invalid operand	abc	Invalid input; request continues.
T08	Exponentiation	2, 3	8.0
T09	Invalid menu	x	Invalid input message.
T10	Exit	2	Farewell message and termination.

11. Challenges Faced

- Managing multiple nested input loops while preserving a clear return path to the main menu.
- Handling malformed numeric input without terminating the application.
- Preventing division by zero during sequential division.
- Defining sensible behavior for empty operand lists.
- Maintaining correct Python indentation across nested loops and conditional branches.
- Ensuring exponentiation and exit choices are not accidentally scoped inside the basic arithmetic branch.

12. Learnings & Key Takeaways

- Functions make repeated arithmetic behavior easier to understand and test.
- Input validation is essential in interactive programs because users do not always follow the expected format.
- Loop structure determines both usability and control flow.
- Error messages should explain what went wrong and what the user should do next.
- Python indentation is part of program structure, not merely formatting.
- A small program can still benefit from requirements, architecture, diagrams, and systematic tests.

13. Future Enhancements

- Correct and verify indentation and add automated tests for every function.
- Add modulus, floor division, square root, logarithms, trigonometric functions, and factorial.
- Support mathematical expression parsing with operator precedence.

- Add a graphical or web interface.
- Provide calculation history and an optional persistent database.
- Add configurable rounding and formatted output.
- Improve menu wording, spelling, and prompts.
- Use a `main()` function and `if __name__ == "__main__":` guard for better reuse and testing.
- Add logging and structured exception handling for unexpected runtime errors.

14. References

1. Supplied project source: *vityarhi-pro1.py*, provided with the assignment materials. The implementation description, requirements mapping, and diagrams in this report are derived from that source.
2. Python documentation concepts used by the program: functions, lists, loops, conditional statements, input/output, float conversion, exception handling, and arithmetic operators. Consult the official Python documentation when preparing the final implementation-specific bibliography.
3. Course or institutional software-project-report guidelines, if applicable. Add the prescribed citation style, author details, institution, and submission date before final submission.